NeutronTask: Scalable and Efficient Multi-GPU GNN Training with Task Parallelism

Zhenbo Fu, Xin Ai, Qiange Wang, Yanfeng Zhang, Shizhan Lu, Chaoyi Chen,
Chunyu Cao, Hao Yuan, Zhewei Wei, Yu Gu, Yingyou Wen, Ge Yu
Northeastern University, China; National University of Singapore, Singapore;
Renmin University of China; Neusoft AI Magic Technology Research

Introduction

- Researchers have proposed multi-GPU processing and data parallelism to meet the resource-intensive requirements of large-scale GNN training
- Data parallelism partitions the input graph to enable parallel computation across multiple GPUs and handle remote neighbor aggregation by caching neighbor replicas

Partitions	1	2	4	8	16	32	64
NR	1	1.25	1.52	2.13	3.02	4.46	6.34
MC (/GB)	335.95	380.22	428.03	536.05	693.65	948.65	1281.56

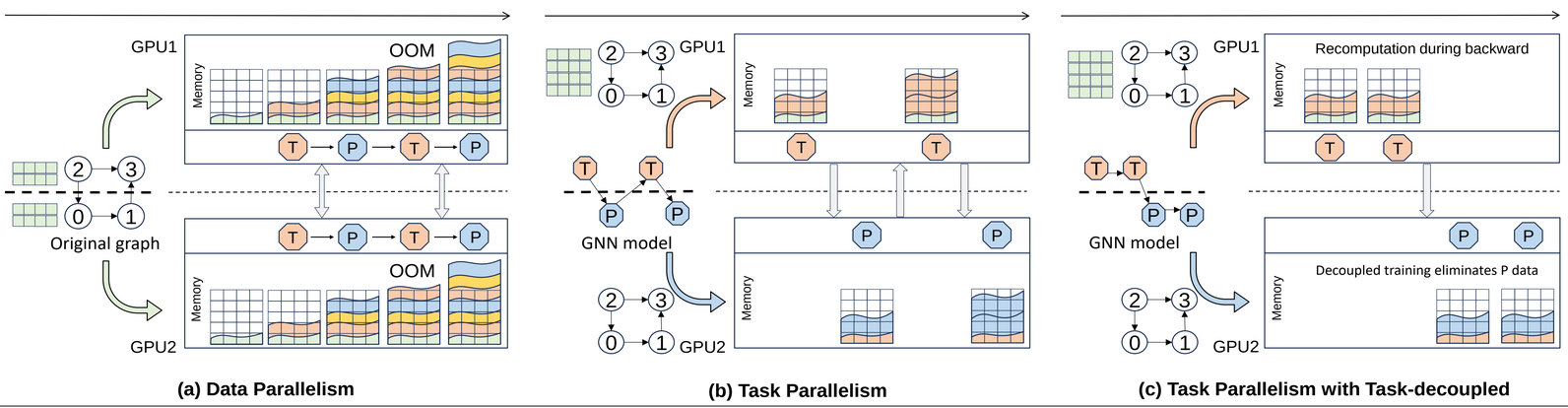
Neighbor Replication

Dataset	#hidden	Params	Topo	Feat	Intr
ogbn-products	256	0.4MB	0.24GB	0.9GB	10.68GB
ogbn-papers100M	128	0.25MB	6.4GB	52.97GB	335.95GB

Intermediate Data

- Multi-GPU processing struggles with large graphs due to limited memory versus high consumption
- Neighbor replication refers to each GPU creating replicas of neighbors from remote GPUs
- Intermediate data refers to the inputs and outputs of T and P tasks for each GNN layer, cached during forward propagation for gradient computation in backward propagation

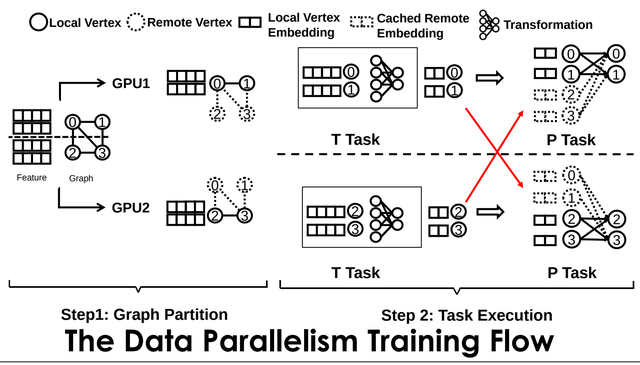
- Our Contribution: NeutronTask
  - Propose GNN task parallelism, which reduces neighbor replication by partitioning tasks across GPUs and partitioning graph of each task within a GPU, instead of partitioning graph data
  - Propose a task-decoupled GNN framework that decouples T and P tasks. utilize the recomputation technique and flexible resource allocation to improve memory efficiency



(a) Data Parallelism (b) Task Parallelism (c) Task Parallelism with Task-decoupled

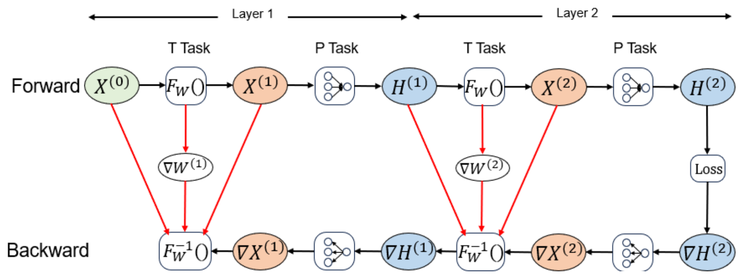
Existing Multi-GPU GNN Systems with Data Parallelism and Their Limitations

- P tasks require access to neighbors on remote GPUs, leading to vertex dependencies (the dashed vertices)
- To manage these dependencies, data parallelism fetches remote vertex data, creating local replicas to facilitate local computation



The Data Parallelism Training Flow

- In forward propagation, GNN computes vertex embeddings by alternately executing T and P tasks
- Only T tasks contain NN parameters
- It is precisely the alternating execution that requires even the linear P tasks to store intermediate data



The Data Flow of 2-layer GCN

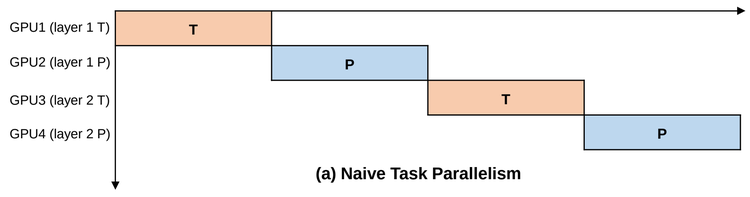
- The major sources of high memory consumption are neighbor replication and intermediate data
- Neighbor replication stems from inter-GPU graph partition
- The alternating execution of tasks is the primary reason for the large proportion of intermediate data

High Memory Consumption ⇐ Neighbor Replication ⇐ Inter-GPU Graph Partition
High Memory Consumption ⇐ Intermediate Data ⇐ Task Alternating Execution

The limitations of existing systems

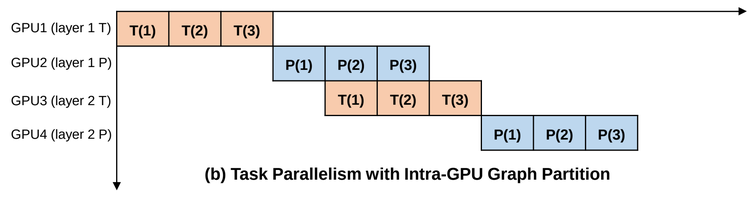
GNN Task Parallelism

- Task Partition: partition the T and P tasks of each model layer across different GPUs, limiting each GPU's memory usage to the intermediate data of the allocated tasks, rather than managing all tasks and neighbor replications
- Initial partitioning strategies:
  - $N_{GPU} < N_{task}$: Each GPU handles T or P task, maximizing resource utilization
  - $N_{GPU} = N_{task}$: Each GPU handles a single task
  - $N_{GPU} > N_{task}$: Hybrid parallelism approach

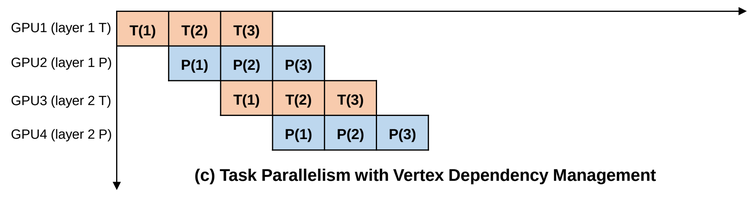


(a) Naive Task Parallelism

- Intra-GPU Graph Partition: address situations where the intermediate data generated by the allocated tasks exceeds a single GPU memory capacity

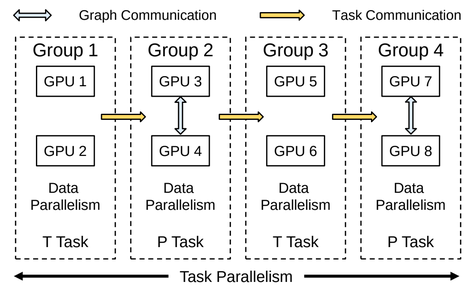


(b) Task Parallelism with Intra-GPU Graph Partition



(c) Task Parallelism with Vertex Dependency Management

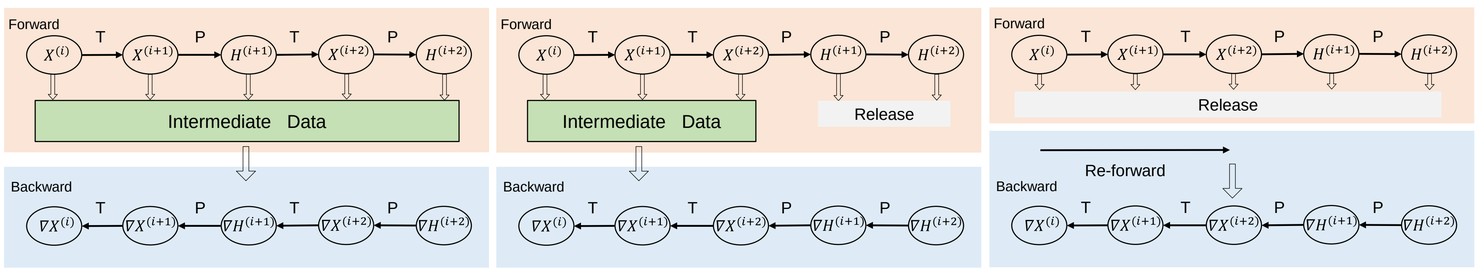
- Vertex Dependency Management: Neighbor cache and reuse; subgraph reorganization before each epoch; overlap communication and computation



- Hybrid parallelism: groups GPUs, with data parallelism within groups and task parallelism between groups

Task-Decoupled GNN Training Framework

- Task-decoupled training: The P tasks are extracted from the GNN model and executed consecutively after all T tasks. The intermediate data generated by P tasks can be immediately released
- Recomputation: Based on task decoupled execution, T tasks are grouped together for execution, which can be seen as DNN training. The recomputation can be used to reduce the intermediate data generated by T tasks



(a) Original GNN Training (b) Task-decoupled GNN Training (c) Task-Decoupled GNN Training with Recomputation

Experiments

- Server 2 Intel(R) Xeon(R) Silver 4316 CPUs, 377GB DRAM, and 4 NVIDIA A5000 GPU (24GB)
- Dataset

Dataset	|V|	|E|	#F	#L	|TR|
cora [43]	2.70K	5.43K	1433	7	59.3%
reddit [21]	232.96K	114.62M	602	41	90.6%
ogbn-products [24]	2.45M	61.86M	100	47	8.03%
it-2004 [3]	41M	1.2B	256	64	25%
ogbn-papers100M [24]	111.06M	1.62B	128	172	1.1%
friendster [29]	65.6M	2.5B	256	64	25%

- Compared approaches: **DGL** [ICLR'19], **Sancus** [VLDB'22], **NeutronStar** [SIGMOD'22]

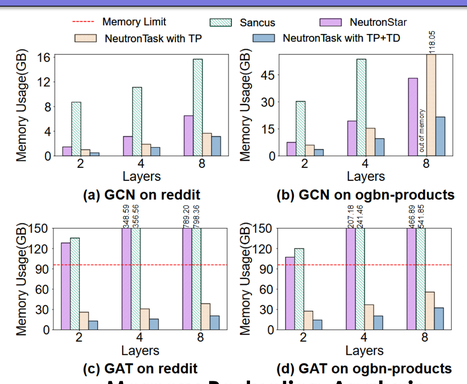
Code Address Personal WeChat

Layers	Dataset	GCN			
		DGL	Sancus	NeutronStar	NeutronTask
2	it-2004	48.77(3.31×)	OOM	OOM	14.72
	ogbn-papers100M	2.76(0.13×)	OOM	OOM	25.88
	friendster	65.6(3.46×)	OOM	OOM	18.98
4	it-2004	64.4(3.45×)	OOM	OOM	18.69
	ogbn-papers100M	17.86(0.36×)	OOM	OOM	57.72
	friendster	OOM	OOM	OOM	35.31
8	it-2004	96.45(2.58×)	OOM	OOM	37.45
	ogbn-papers100M	OOM	OOM	OOM	84.54
	friendster	OOM	OOM	OOM	77.71

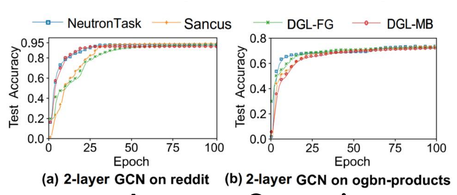
Overall Performance Comparison (Partial Results)

- 1.81X-9.3X faster than DGL
- 1.52X-5.47X faster than Sancus
- 1.58X-3.25X faster than NeutronStar

NeutronTask can support the training of graphs of all sizes, whereas DGL, Sancus, and NeutronStar encountered OOM (Out of Memory) errors under various conditions

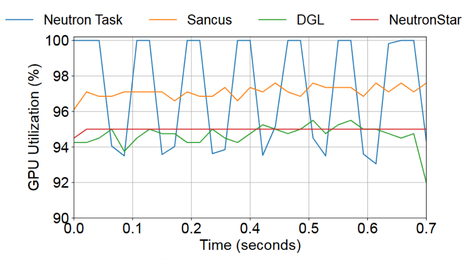


Memory Reduction Analysis



Accuracy Comparison

- NeutronTask reduces total memory consumption by 49% – 67%



GPU Utilization

- High GPU utilization ensures shorter runtime, while the fluctuation is caused by pipeline bubble time
- NeutronTask achieves comparable accuracy with baseline